Contents

S6 Rerank — Every Signal, In Detail **1**
The loop . . . 1
1. coverage — verbatim fragment matches (span_weight = 1.0, length_bonus = 0.12) . . . 2
2. pair_coverage — intact key:value associations (pair_weight = 0.8) . . . 2
3. retrieval_score / top_score — the inherited S5 score (retrieval_weight = 1.0) . . . 3
4. popularity (popularity_weight = 0.02) . . . 3
5. facet_score — structured attribute agreement (facet_weight = 0.3) . . . 3
6. category_score — loose category agreement (category_weight = 0.4) . . . 3
7. tail_score — strict category-tail match (tail_weight = 0.8) . . . 4
8. conflict_score — negative facet evidence (facet_conflict_weight = 0.4, SUBTRACTED) . . . 4
depth = 300 . . . 4
How weights are chosen — the house method . . . 5
Not a signal: price / budget . . . 5
Not a signal: `user_profile` . . . 5
Status . . . 5
Why "more information -> large weight" is the wrong instinct here . . . 6
Where the profile *could* carry non-redundant signal . . . 6

S6 Rerank — Every Signal, In Detail

Reference for `src/rerank.py` as of `96513ba` — still the most recent commit to touch that file. Covers the eight scoring signals, exactly how each is computed, and how each weight was chosen. Closes with the one agent input that is *not* a signal — `user_profile` — and what could be done with it.

This doc is the as-built spec: what the reranker does today. Its companion `rerank_signals.md` is the decision log: what was tried, what the measurements said, and what was rejected — including three signals that never shipped (profile-weighted agreement, budget/price closeness, fragment de-weighting) and one piece of current behaviour that was challenged and upheld. Where a weight is justified here in a sentence, the full ablation lives there. Keep them in step: a signal added or a weight changed needs an edit in both.

The loop

```
spans = state.query_spans()
if not spans:                          # nothing disclosed yet -> rerank is a NO-OP;
    return candidates                  # the pool passes through in RRF order
head = candidates[:300]                # depth == pool_size, so the whole pool
pairs = state.query_pair_spans() if config.pair_weight else []   # skipped when off
top_score = max(retrieval_score in head) or 1.0
customer_facets      = extract_query_facets(state.full_text())      # whole convo
authoritative_facets = extract_query_facets(state.focused_text())   # post-override only

for (asin, retrieval_score) in head:
    p = index.products[asin]           # {text, categories, store, price, ratings}
    total = ( 1.00 * coverage
            + 0.80 * pair_coverage
            + 1.00 * (retrieval_score / top_score)
            + 0.02 * popularity
            + 0.30 * facet_score
            + 0.40 * category_score
            + 0.80 * tail_score
```

```
            - 0.40 * conflict_score )
sort by (-total, asin);  return scored + tail
```

`product["text"]` is the token-joined, lowercased blob of `title + features + description + details`, built once at index time. Every signal reads that string or the trimmed product dict. **No FTS5, no bm25() — the only retrieval quantity used is the pre-computed fused score.**

Key gate: **before a real fragment is disclosed (turn 1, or a customer who has only declined), `rerank()` returns the pool untouched.** Ordering before then is pure S5.

That gate is deliberate and was challenged on the reasonable grounds that "no spans" is not "no information" — the opening always names a category. It was measured and kept (`rerank_signals.md` §5): the gate only affects a turn that actually shows a list in 3.4% of public turns, the buying opening's hard requirement is a single word so no span exists to extract either way, and the best variant moved 4 sessions of 200 while regressing the override bucket.

---

1. coverage — verbatim fragment matches (span_weight = 1.0, length_bonus = 0.12)

Input `state.query_spans()`: every customer message on turn >= 2 that is not a decline, split on `. ; : , \n` and `" - "`; each chunk tokenised to `[a-z0-9]+`, lowercased, single-space joined; kept if 2-25 words and not all-stopwords; newest-first, deduped. Example turn 2 -> `["100 cotton", "heather grey", "90 cotton", "10 polyester", "solid colors", ...]`

Computation For each span, a word-bounded substring test: `f" {span} " in f" {product_text} "`. On a hit: `coverage += 1.0 + 0.12 * word_count`. A 2-word match = 1.24; a 5-word match = 1.60.

Role The dominant term. The customer's constraints are copied verbatim from the target's own metadata, so literal phrase containment is a near-identity signal. A target matching 8 fragments scores ~11; every other term maxes ~1-2.

Weight Reference — span coverage is the unit everything else scales against, so `span_weight = 1.0` by definition. `length_bonus = 0.12` is a hand-set nudge, never rigorously swept. An alternative (weight spans by pool-local rarity) was measured at +0.0002 dev / 0 holdout and deleted.

---

2. pair_coverage — intact key:value associations (pair_weight = 0.8)

Input `state.query_pair_spans()`: like `query_spans` but splits **only** on `. ; \n` and `" - "` (keeps colon- and comma-joined content together), strips a leading run of stopwords, minimum 3 tokens, and drops anything already emitted as a fragment span (no double-count). Example -> `"heather grey 90 cotton 10 polyester"` as one unit.

Computation `pair_coverage += 1.0` per pair span found (word-bounded). **Flat** — no length bonus; the pair's evidential value is the intact association, not its length.

Role A fragment asks "does this product mention 90% cotton at all?"; the pair asks "does it say that *about heather grey*?" It separates homogeneous-cluster bucketmates that pair the same compositions differently — the case where every fragment matches for everyone.

Weight Swept `pair00 / pair08 / pair15` (0.0 / 0.8 / 1.5). "0.4-1.5 score identically on dev (0.9233) and holdout (+0.0008); 0.8 sits mid-plateau." Turning it on: public 0.9149 -> 0.9159, hard 0.7917 -> 0.7944, with the entire gain in `homogeneous_cluster` (MRR 0.431 -> 0.478).

---

3. retrieval_score / top_score — the inherited S5 score (retrieval_weight = 1.0)

Input `retrieval_score` is the RRF-fused number from S5 (`sum over routes of weight / (60 + position + 1)` across the anchor / terms / focused routes). Derived from BM25 *rankings*, not BM25 values.

Computation Divided by `top_score` (the max in the head) -> scaled to `(0, 1]`. Weighted 1.0.

Role "How much did retrieval like this overall" — a soft prior and a backstop for candidates whose span coverage is weak. In practice dominated by `coverage` for well-matched candidates; effectively a tie-break for the rest.

Weight Reference — inherited from the original three-term formula (`span + normalised-retrieval + popularity`). Never independently swept; kept at parity with the span-coverage unit.

---

4. popularity (popularity_weight = 0.02)

Computation `(avg_rating / 5.0) * min(1.0, log10(rating_count + 1) / 4.0)`, bounded `[0, 1]`. 5 stars with 10,000 reviews -> 1.0; 4 stars with 100 reviews -> 0.4.

Role Pure tie-break. At weight 0.02 it can only reorder candidates already equal on every other term.

Weight Deliberately near-zero **by reasoning, not sweeping**: the target is one specific person's purchase, not a bestseller, so a strong popularity prior would systematically drag the ranking toward famous products and away from the answer.

---

5. facet_score — structured attribute agreement (facet_weight = 0.3)

Input - `customer_facets = extract_query_facets(full_text())`: regex the whole (non-declined) conversation for the first vocabulary term in each of `{material, color, style, use_case, size}` -> e.g. `{material: cotton, color: grey}`. - `product_facets = extract(product)`: the same 5 regex over `product["text"]`, plus brand / budget / category (never present in `customer_facets`, so never match).

Computation `facet_score = |{k : customer_facets[k] == product_facets[k]}|`. Max 5, realistically 1-2.

Role Catches agreement that literal span matching misses. Note it **saturates** inside a homogeneous cluster — every bucketmate extracts the same one or two values — which is what motivated signals 6-8.

Weight `0.3` ~= a quarter of one span match: it nudges, it never overrides `coverage`. Measured effect when introduced: public MRR +0.011.

---

6. category_score — loose category agreement (category_weight = 0.4)

Input `opening_terms = set(terms(state.opening, drop_boilerplate=True))` — the content tokens of the turn-1 message, e.g. `{novelty, women}` from "I'm looking for Novelty Women".

Computation For each value in the product's category path, tokenise it; if it shares **any** token with `opening_terms` -> `+1.0`.

Role Broad "is this roughly the right department". Ancestor-inclusive, so it **saturates**: a deep competitor under `... > Novelty > Women > Tops & Tees > T-Shirts` scores the same as the shallow target `... > Novelty > Women`.

Weight `0.4`; the first (weaker) half of the category fix — the tail term below is what actually discriminates.

---

7. tail_score — strict category-tail match (tail_weight = 0.8)

Input the same `opening_terms`.

Computation Drop generic wrappers ("clothing", "clothing shoes & jewelry") from the product's categories, take the **last 2** ("the tail"). For each: if **every** token of that level is a subset of `opening_terms` -> `+1.0`. (Subset, not intersection.) Target `... > Novelty > Women`: tail `["Novelty","Women"]`, both subset of `{novelty, women}` -> `2.0`. Deep competitor: tail `["Tops & Tees","T-Shirts"]` -> `0.0`.

Role The evaluator builds the opening line from the target's two most-specific category levels, so **only candidates on the right leaf score here** — the discriminator when spans, facets and `category_score` all saturate.

Weight Swept: "0.6-1.5 score identically on dev and holdout; this sits mid-plateau." Measured effect: public 199/200 -> 200/200, score 0.9029 -> 0.9125, adversarial 0.789 -> 0.8007; the one public miss (`public_0020`) went from rerank rank 171 into the top 10. Full write-up: `category_tail_match.md`.

Those adversarial figures are **pre-anchor-retrieval-merge** and are not comparable to the ones quoted for signals 2 and 8: that merge moved the hard-set baseline from 0.8007 down to 0.7914. Only compare hard-set numbers measured in the same session as each other.

---

8. conflict_score — negative facet evidence (facet_conflict_weight = 0.4, SUBTRACTED)

Input `authoritative_facets = extract_query_facets(focused_text())` — **post-override turns only** (identical to `full_text()` until an override fires).

Computation `_facet_conflicts()`: for each stated `(key, value)`: 1. skip if `key not in product_facets` — silence is not disagreement; 2. build aliases (`grey <-> gray` is the only pair; else just the value); 3. skip if any alias appears word-bounded (`\bword\b`) in `product["text"]` — guards multi-value products ("black/grey reversible" extracts `color: black` but contains "grey"); 4. otherwise `conflicts += 1.0`. `total -= 0.4 * conflict_score`.

Role The only term that can **rule a candidate out**: a shirt that resolves a colour and never says "grey" loses 0.4 when the customer wants grey. Positive signals can't do this — a black shirt matches "cotton shirt" spans exactly as well as a grey one.

Weight Swept `conflict00 / 04 / 08`. "dev 0.9224 at 0.4 and 0.9226 at 0.8 — a plateau, and a penalty term gets the smallest weight on it." Also a deliberate ceiling: at 0.4 one conflict cannot overturn a genuine span lead (each span match is worth >= 1.12). Had a bug — judged against `full_text()` it punished override targets for obeying the reversal (`generic_override` MRR 0.673 -> 0.626); the `focused_text()` fix restored it and is unit-tested.

---

depth = 300

`head = candidates[:300]` — only the head is rescored. Was `200`, raised because "~12% of cluster-target sessions had the target in the pool but past rank 200, where the span signal never applied." With `pool_size = 300` the tail is now empty, so the whole pool is reranked.

How weights are chosen — the house method

1. **Reference weights** (`span_weight`, `retrieval_weight` = 1.0) — definitional, not tuned. Span coverage is the unit.
2. **Add an ablation row to `tools/sweep.py build_configs()`** bracketing the candidate weight (`conflict00/04/08`, `pair00/08/15`), then run `python3 tools/sweep.py --split dev` and `--split holdout`.
3. **Find the plateau, pick the middle.** Stated explicitly for `tail_weight`, `pair_weight`, `facet_conflict_weight`. A split's argmax "is how you buy noise"; a knife-edge (best at one value, monotone worse either side) is the signature of overfitting.
4. **Penalty terms take the smallest weight on the plateau**, bounded below one span match, so negative evidence adjusts ties without overriding positive evidence.
5. **Priors are set by reasoning, not sweeping** (`popularity_weight = 0.02`).
6. **Disqualifying check:** the public hit rate must stay 200/200 — a false signal that demotes a true target shows up there first.
7. **Dead options are deleted, never kept at weight 0** (pool-local rarity, profile-weighted agreement — both removed entirely).

Not a signal: price / budget

`product["price"]` is loaded into the trimmed product dict and bucketed by `extract(product)` into a `budget` facet — but no scoring term reads it, and the customer's `"budget around $X"` can never match one: `extract_query_facets` never emits `budget`, and the price is absent from `product["text"]` so no span can match it either.

That is not an oversight. A closeness term was designed and rejected **before implementation** on measurement (`rerank_signals.md` §4): `intent_card()` appends the budget line after every feature/detail candidate and keeps only the first four, so a budget surfaces for **0 of 200 public sessions** and 0.45% of the catalog. The four hard-set sessions carrying one render it as `budget around $-` with no number, and are the four `never_retrieved` misses no rerank term can reach. The design is on file there if a future evaluator emits budgets routinely.

Not a signal: `user_profile`

Status

`reset()` stores `user_profile` in `DialogState.profile`. In the **shipped path** (`FixedPolicy` + retrieve + rerank) **nothing reads it.** The only consumer is `InfoGainPolicy._answerability()` (S4), which nudges *which attribute to ask about* using `preference_tags` — and `InfoGainPolicy` does not ship.

So the reranker has no profile term today. It was tried:

Profile-weighted facet agreement (`rerank_signals.md` section 3): extra credit when a facet agreement lands on an attribute the profile flags via `TAG_HINTS`.

config	dev	holdout
baseline	0.9224	0.9021
profile_weight 0.1	0.9237	0.9013
profile_weight 0.2	0.9217	0.9005
profile_weight 0.4	0.9217	—

A dev gain at 0.1 that does not reproduce on holdout, monotonically worse as the weight grows — a knife-edge, not a plateau. Removed.

Why "more information -> large weight" is the wrong instinct here

The profile is **redundant with the transcript.** Both the profile and the hidden intent card are generated from the same source (the target product / the user's history), and the simulated customer **discloses the intent card during the session.** A customer whose profile says `material` *states the material out loud* by turn 2-3, and `facet_score` already scores it. Re-weighting that same agreement adds variance without adding information.

Weight is set by measurement, not by how much data is on hand. Extra *available* information is not the same as extra *useful* information, and the profile-weight sweep is the direct evidence: bigger weight, worse holdout.

Where the profile *could* carry non-redundant signal

Each session hides ~4 constraints and the customer discloses at most 2 per answered question; the profile has up to 3 tags plus two rating fields. The narrow openings:

1. **`average_prior_rating` / `rating_style` modulating popularity.** Both fields are read by nothing. A "critical" reviewer with average 1.0 plausibly buys different products than a 5.0 "usually positive" one, and the customer never says this out loud, so it is genuinely orthogonal to the transcript. Concrete: scale (or flip the sign of) `popularity_weight` by the profile — do not boost popular items for a low-rating critical buyer. Expected value low-moderate; rating style is a weak identity signal, but it costs one cheap term and uses two dead fields. Sweep it as a `popularity_profile_weight` ablation.
2. **Tags for a dimension the conversation never covers.** If a tag names an attribute the agent never got to ask about (ladder ran out, or the target has no constraint of that type), the tag is the only evidence for that dimension. Boost candidates whose facets are plausible for that tag. Narrow — fires on maybe a handful of sessions — but it is the one place the profile is not redundant by construction.
3. **Tag-driven marketing-language match inside a saturated cluster.** Where spans / facets / category all tie (the `public_0020` situation), match the tag concept against *copy*, not structured facets: `durability` -> `{durable, heavy duty, reinforced, rugged}`, `comfort` -> `{soft, breathable, lightweight, cozy}`. Only fires as a last-resort tie-break. Expected value low — cluster members share their marketing copy too — but it is cheap and only active where everything else has failed.
4. **Ship a small profile term as private-set insurance.** If the organizer's private simulator paraphrases more heavily or discloses fewer constraints, the redundancy breaks and the profile becomes a real fallback. A ~0.1 term that is flat on the public set but non-negative on holdout is defensible as robustness even though house convention is to delete dead options.

Recommendation. Try option 1 (popularity modulation) first — it is the only one touching genuinely unused, genuinely orthogonal fields. Expect it to be flat-to-slightly-positive; keep it only if holdout agrees, at the smallest weight on its plateau. Options 2-3 are last-resort tie-breaks worth a single sweep each. Do not give any profile term a large weight — the evidence says it overfits.